ForecastLabAI — 12 interview questions with short answers

Problem

Rehearse the questions most likely to come up about this project, with answers that match the real code.

Instant solution

1. **Forecasting vs regression?** Regression maps features to a target. Forecasting predicts future values, so time order, distribution shift, horizon and future feature availability matter.
2. **Why baselines first?** They set a performance floor. If a complex model can't beat seasonal naive on representative backtests, its cost isn't justified.
3. **What is data leakage?** Using information in training/evaluation that wouldn't exist at prediction time. Prevented with chronological cutoffs, `shift(1)` features and fold separation.
4. **Why not shuffled K-fold?** It mixes past and future. Expanding/sliding windows keep time order like real deployment.
5. **Random forest vs boosting?** RF: independent trees averaged (less variance). Boosting: sequential trees correcting previous errors.
6. **RAG vs fine-tuning?** RAG injects retrieved knowledge at inference; fine-tuning changes weights. ForecastLabAI uses RAG; they can be combined.
7. **What is an embedding?** A vector representing meaning so similarity is measurable; both chunks and queries are embedded.
8. **Role of pgvector?** Vector type + similarity search (HNSW, cosine) inside PostgreSQL, next to the relational data.
9. **Why PydanticAI?** Typed deps (`AgentDeps`), registered tools, output schemas — agents fit into normal Python services.
10. **Agent vs chatbot?** A chatbot answers; an agent chooses tools, inspects results and takes bounded steps, under app-defined controls (typed tools, human approval for state changes).
11. **Does async make training non-blocking?** No. Async helps I/O; CPU-bound training can still block the event loop → workers.
12. **When is a model deployment-ready?** Beats baselines in backtests, acceptable stability and bias, valid inference inputs, reproducible artifact (hash, registry), passing operational checks — not just the lowest error.

Why it works / gotchas

- Answer in three levels: business problem → implementation → engineering decision.
- Use "I" only for work you personally did.
- Deeper notes: leakage, backtesting, RAG, agents, DevOps.